# One full VAE training step, by hand

Every weight written down, every derivative computed, then re-run to prove the loss falls. No step skipped.

| | | | |
|---|---|---|---|
| data | **x = 2.0** | encoder | **μ = wμx + bμ,  logσ² = w_v x + b_v** |
| weights | wμ=0.5  bμ=0  w_v=0.2  b_v=−0.5 | decoder | **x̂ = w_d z + b_d**  w_d=0.8, b_d=0.1 |
| noise | ε = 0.3  (drawn from N(0,1)) | reparam | **z = μ + σ·ε**  learning rate α = 0.01 |

## FORWARD PASS

**1 encode — the mean**

μ = wμ·x + bμ = 0.5(2.0) + 0 = **1.000000**

**2 encode — the log-variance**

logσ² = w_v·x + b_v = 0.2(2.0) − 0.5 = **−0.100000**

**3 variance**

σ² = e^(−0.1) = 0.904837

**4 standard deviation**

σ = √0.904837 = **0.951229**

**5 reparameterize (ε is an INPUT, not a layer)**

z = μ + σ·ε = 1.000000 + 0.951229(0.3)
= 1.000000 + 0.285369 = **1.285369**

**6 decode**

x̂ = w_d·z + b_d = 0.8(1.285369) + 0.1
= 1.028295 + 0.1 = **1.128295**

**7 reconstruction term**

recon = (x − x̂)² = (2.0 − 1.128295)²
= (0.871705)² = **0.759869**

**8 KL term**

KL = ½(μ² + σ² − 1 − logσ²)
= ½(1.000000 + 0.904837 − 1 + 0.100000)
= ½(1.004837) = **0.502419**

**9 loss**

loss = recon + KL = 0.759869 + 0.502419
= **1.262288**

### READ THE FORWARD RESULT

$\hat{x}$ = 1.128 is **well below** x = 2.0, so the model is badly **undershooting**.

That is why **every** gradient in the right-hand column turns out negative: raising w_d, raising b_d, raising μ (→ raises z), raising wμ — every one of them pushes $\hat{x}$ *upward*, toward the target. The signs all agree because the error has one clear direction.

## BACKWARD PASS

**1 the sum splits**

∂L/∂recon = 1  ∂L/∂KL = 1

**2 local derivative of squared error**

∂recon/∂x̂ = −2(x − x̂) = −2(0.871705)
= **−1.743410**

**3 ∇ to decoder weight (∂x̂/∂w_d = z)**

∇w_d = −1.743410 × 1.285369 = **−2.240925**

**4 ∇ to decoder bias**

∇b_d = −1.743410 × 1 = **−1.743410**

**5 gradient heading toward z (∂x̂/∂z = w_d)**

dz = −1.743410 × 0.8 = **−1.394728**

**6 KL → μ**

∂KL/∂μ = μ = **1.000000**

**7 KL → logσ²**

∂KL/∂logσ² = ½(σ² − 1) = ½(−0.095163) = **−0.047581**

**8 reparam node's local derivatives**

∂z/∂μ = 1  ∂z/∂σ = ε = 0.3
∂σ/∂logσ² = ½σ = 0.475614

**9 ACCUMULATE AT μ — the two forces in one line**

∂L/∂μ = dz(1) + ∂KL/∂μ
= −1.394728 + 1.000000
↑recon  ↑KL
= **−0.394728**

KL cancelled 72% of reconstruction's pull.

**10 accumulate at logσ²**

∂L/∂logσ² = dz·ε·½σ + ∂KL/∂logσ²
= (−1.394728)(0.3)(0.475614) − 0.047581
= −0.199006 − 0.047581 = **−0.246587**

**11 ∇ to encoder weights (∂μ/∂wμ = x = 2.0)**

∇wμ = −0.394728 × 2 = **−0.789456**
∇w_v = −0.246587 × 2 = **−0.493174**

**12 ∇ to encoder biases**

∇bμ = **−0.394728**  ∇b_v = **−0.246587**

## ONE UPDATE  PARAM ← PARAM − A·∇  (A = 0.01)

| param | old | ∇ | new |
|---|---|---|---|
| w_d | 0.8 | −2.240925 | 0.822409 |
| b_d | 0.1 | −1.743410 | 0.117434 |
| wμ | 0.5 | −0.789456 | 0.507895 |
| bμ | 0.0 | −0.394728 | 0.003947 |
| w_v | 0.2 | −0.493174 | 0.204932 |
| b_v | −0.5 | −0.246587 | −0.497534 |

## PROOF — RE-RUN THE FORWARD PASS

μ = 1.019737  logσ² = −0.087670
σ = 0.957101  z = 1.306868  x̂ = 1.192223

| recon | KL | loss |
|---|---|---|
| **0.652515** | **0.521798** | **1.174313** |
| ▼ from 0.759869 | ▲ from 0.502419 | ▼ from 1.262288 |

**loss fell by 0.087975 ✓**

**KL went UP while loss went DOWN.** The encoder deliberately made its code less prior-like to make it more informative. Not a bug — that *is* the algorithm, and it is the same signature the 12,000-step simulation showed.

**Step 9 is the whole VAE in one line.** Reconstruction pulls μ down with −1.395; KL pulls it back with +1.000. If μ were near 0, KL's term would vanish — which is exactly why collapsed dimensions are the ones sitting at μ ≈ 0.